

Method Used	Dataset Size	Testing-set predictive performance	Time taken for the model to be fit
XGBoost in Python via scikit-learn and 5-fold CV	100		
	1000		
	10000		
	100000		
	1000000		
	10000000		
XGBoost in R – direct use of xgboost() with simple cross-validation	100	0.9	0.0131s
	1000	0.96	0.0274s
	10000	0.975	0.19275s
	100000	0.9844	1.5920491s
	1000000	0.98861	14.8400612s
	10000000	0.99215	102.568s
XGBoost in R – via caret, with 5-fold CV simple cross-validation	100	0.94	1.98s
	1000	0.96202	4.84s
	10000	0.9831	25.96s
	100000	0.9901	241.957s
	1000000	0.9906	985.263s
	10000000	0.9845	1985.765s

2.

The application of direct XGBoost using simple cross-validation yields better results than implementing it through caret. Using a direct implementation of XGBoost delivers faster execution speeds alongside equivalent or even better forecasting performance during analyses of larger datasets.

The direct XGBoost implementation achieves better scalability for increasing dataset sizes according to performance metrics. The direct implementation outpaces the caret approach by 19

times when processing the 10M record dataset with a performance speed of 102.5 seconds compared to 1985.7 seconds. The substantial surge in speed becomes crucial for rapid retraining needs and massive dataset applications that demonstrate significant operational effects in the time difference between seconds and hours. Using direct implementation offers superior performance benefits that outweigh the unified interface convenience provided by caret when operating on datasets which exceed a few thousand records.